CPSY 1291 · Recitation handout 6

Held-out data, and images through a network

Splits, overfitting curves, error bars — and what a convolution actually does

Fall 2026 · Week 6 · hold this after Lecture 10; Assignment 3 is due Thu 11/5

What this session is for

This week carries two jobs. Exam 1 is Thursday of next week and Recitation 7 is given over to it entirely, so the mechanics Assignment 3 needs arrive here, before the assignment is released on Thursday 10/22.

Part A is the discipline of held-out data — the three splits, the diagnostic plot of the whole course, and the small amount of probability needed to put an error bar on anything. It is the most transferable material in the term: it applies to a psychology experiment as directly as to a network.

Part B is images. What a convolution computes, how to predict the shape that comes out of one, and why weight sharing is the whole reason vision models work. Lecture 11 covers the biology and the architecture; this covers the arithmetic, a few days early, so that the assignment is not blocked.

> **The one idea.** A number computed on data your model has already seen is not a measurement of your model. It is a measurement of your model's memory. Every practice in Part A exists to keep those two things apart.

Part A — Held-out data

1 Three splits, three jobs

Split	Used for	Seen by the model
training	fitting the parameters	constantly
validation	choosing hyperparameters, early stopping	indirectly, many times
test	one final estimate of generalization	*once*, at the very end

The distinction between the second and third row is the one people get wrong. Validation data is not seen by the optimizer, but it *is* seen by you — every time you look at validation accuracy and change something, a little information leaks from the validation set into your model. Do that twenty times and validation accuracy is optimistic too.

The cardinal rule. The test set is looked at once, after every decision has been made. If you tune anything against the test set, you no longer have an estimate of generalization — you have a training score with extra steps. This is the most common methodological error in student projects, in papers, and in industry.

```
from sklearn.model_selection import train_test_split

Xtr, Xte, ytr, yte = train_test_split(
   X, y, test_size=0.2, random_state=0, stratify=y)
```

`stratify=y` keeps the class proportions the same in both halves. Without it a small or imbalanced dataset can give a split whose test set is missing a class entirely, and the resulting accuracy is not comparable to anything.

1.1 When the data is small: cross-validation

Splitting off 20% of 200 examples leaves a test set of 40, and an accuracy estimated from 40 examples has a standard error of several percentage points. Cross-validation reuses the data: split into k folds, train on $k-1$, test on the held-out one, rotate, average.

```
from sklearn.model_selection import cross_val_score
scores = cross_val_score(clf, X, y, cv=5)
print(f"{scores.mean():.3f} +/- {scores.std():.3f}")
```

Report the spread as well as the mean. A model scoring 0.72 ± 0.02 and one scoring 0.75 ± 0.09 are not distinguishable, and quoting only the means would say otherwise.

The leak that hides here. Any preprocessing fitted on the data — scaling, PCA, feature selection — must be fitted *inside* each fold, on the training part only. Fit PCA on all your data and then cross-validate and every fold's "held-out" data has already influenced the features. The scores go up. They are wrong. Use `sklearn.pipeline.Pipeline`, which does this correctly by construction.

2 The diagnostic plot of the course

Plot training error and validation error against something that increases model capacity — epochs, hidden units, parameters — on the same axes. Four shapes, four diagnoses:

What you see	What it means
both high, both flat	**underfitting** — too little capacity, or too small an η
train falls, val follows	healthy; keep going
train $\to 0$, val turns up	**overfitting** — stop at the val minimum
train high, val *lower*	a bug: leakage, or a mismatched split

```
plt.plot(train_acc, label='train')
```

```
plt.plot(val_acc, label='validation')
plt.legend()
```

> **Overfitting is really about too few examples.** A model with a million parameters is not overfitting because a million is a large number — it is overfitting because it has more freedom than the data constrains. The same model on ten times the data may not overfit at all. Whenever you are tempted to say "the model is too big", check whether you mean "the dataset is too small".

3 Double descent, plotted honestly

The classical picture says test error follows a U as capacity grows: it falls, then rises once the model can memorize. That picture is incomplete. Keep growing the model past the point where it fits the training data exactly — the **interpolation threshold** — and test error frequently falls again, often below the bottom of the U.

```
plt.plot(widths, test_err, 'o-')
plt.xscale('log') # you cannot see it on a linear axis
plt.axvline(w_interp, ls=':')
```

Two practical notes, both of which decide whether you see the effect at all:

- **Use a log x-axis.** The interesting widths span orders of magnitude, and on a linear axis the entire first descent is compressed against the origin.
- **The peak is at the interpolation threshold**, roughly where the number of parameters matches the number of training examples — so you have to sweep widths that bracket it. Sample too coarsely and you get a curve that merely looks noisy.

> **What double descent does and does not claim.** It says the classical capacity–generalization bargain is not the whole story, and that "bigger overfits more" is false past a point. It does *not* say overfitting is imaginary, and it does not license training without a validation set.

4 Enough probability to put an error bar on things

4.1 Four definitions

A **random variable** is a quantity whose value depends on chance — a network's test accuracy, given a random initialization. Its **expectation** $\mathbb{E}[X]$ is its long-run average, and its **variance** $\mathrm{Var}[X] = \mathbb{E}[(X - \mathbb{E}[X])^2]$ is how much it typically departs from that average. The **standard deviation** is $\sqrt{\mathrm{Var}[X]}$, and lives in the same units as X.

4.2 The fact everything rests on

If you draw n independent samples with standard deviation σ and take their mean, that *mean* has standard deviation $\sigma/\sqrt{n}$ — the **standard error**.

```
acc = np.array([run(seed) for seed in range(10)])
print(f"{acc.mean():.3f} +/- {acc.std(ddof=1)/np.sqrt(len(acc)):.3f}")
```

> **Standard deviation or standard error?** They answer different questions. The **standard deviation** says how much a single run varies — report it when the question is "how reliable is this architecture?". The **standard error** says how well you know the average — report it when the question is "is this mean different from that one?". Label your error bars. An unlabeled error bar is uninterpretable, and roughly half of them in the literature are.

The $\sqrt{n}$ is also why quadrupling your seeds only halves your error bar. If a difference is not visible with 10 seeds, 40 will rarely rescue it.

4.3 Which n is it?

The commonest error is counting the wrong things. Ten training runs of one architecture give $n = 10$ *runs*, not $n = 10{,}000$ test images. Runs are what varied; the images were held fixed. Ask what you would have to redo to get another independent number — that is your n.

Part B — Images through a network

5 An image is a tensor

Object	Shape	Note
one grayscale image	(H, W)	
one color image	$(3, H, W)$	channels **first** in PyTorch
a batch	$(N, 3, H, W)$	"NCHW"
a conv layer's weights	$(C_{\text{out}}, C_{\text{in}}, k, k)$	C_{out} filters

Pretrained models were trained on inputs normalized per channel, and expect the same normalization at test time. For torchvision's ImageNet models that is mean $(0.485, 0.456, 0.406)$ and standard deviation $(0.229, 0.224, 0.225)$. Skip it and the model still produces numbers; they are simply not comparable to anything published.

6 Convolution is template matching, repeated

A convolutional layer holds a small filter — say $3 \times 3 \times C_{\text{in}}$ — and slides it over the image. At every position it computes a dot product between the filter and the patch underneath. So each output pixel is the answer to "how well does the pattern in this filter match the image *here*?"

Everything from Recitation 4 applies unchanged: the response is $\|\boldsymbol{w}\|\|\boldsymbol{x}\| \cos\theta$ at each location, the optimal patch is the filter itself, and this is why the first layer's filters can be displayed as images and recognized as edge detectors.

6.1 Weight sharing, and the number that justifies it

The same filter is used at every position. Compare the parameter counts for one layer mapping a $224 \times 224 \times 3$ image to 64 feature maps:

Fully connected, to 64 outputs	$224 \cdot 224 \cdot 3 \cdot 64 \approx 9.6$ million
Convolutional, 11×11, 64 filters	$11 \cdot 11 \cdot 3 \cdot 64 = 23{,}232$

Four hundred times fewer parameters, and — more importantly — the built-in assumption that a feature worth detecting in one part of the image is worth detecting everywhere. That assumption is a claim about vision, and it is approximately true, which is why it works.

6.2 The output-size formula

For an input of size H, filter size k, padding p, stride s:

$$H_{\text{out}} = \left\lfloor \frac{H - k + 2p}{s} \right\rfloor + 1.$$

Setting	Effect	Common choice
$p = (k-1)/2$, $s = 1$	size unchanged	"same" padding, odd k
$s = 2$	halves the size	downsampling inside the conv
max-pool 2×2	halves the size	downsampling as its own layer

```
conv = nn.Conv2d(in_channels=3, out_channels=64, kernel_size=3, padding=1)
conv(torch.zeros(1, 3, 32, 32)).shape # (1, 64, 32, 32) -- check by running
```

> **Do not compute this in your head.** Build the layer, push a tensor of zeros of the right shape through it, and print the result. It takes one line and it is never wrong, whereas the formula in your head frequently is.

6.3 One block: conv → ReLU → pool

```
block = nn.Sequential(
   nn.Conv2d(3, 32, 3, padding=1), # template matching
```

```
    nn.ReLU(), # keep the matches, discard the mismatches
    nn.MaxPool2d(2), # tolerate small shifts; halve the size
)
```

This is the unit every convolutional network repeats, and it maps onto the simple-cell/complex-cell story from Lecture 11: selectivity from the convolution, tolerance from the pooling. Stacking blocks makes the receptive field of a unit grow with depth — which is why deep units respond to large, complicated things and early units respond to edges.

7 Getting the activations out of the middle of a network

For most of what you will do, the simplest approach is enough: run the part of the model you want.

```
net = torchvision.models.alexnet(weights='DEFAULT').eval()
with torch.no_grad():
    feats = net.features[:5](x) # everything up to layer 5
feats.shape
```

When the layer you want is not a prefix — or when the model's `forward` does something slicing cannot reproduce — register a **forward hook**, a function the layer calls with its own output:

```
store = {}
def grab(name):
    def hook(module, inp, out): store[name] = out.detach()
    return hook

h = net.features[8].register_forward_hook(grab('conv4'))
with torch.no_grad():
    net(x)
h.remove() # always remove it afterwards
store['conv4'].shape
```

Hooks come back properly in Recitation 9. For now: `.detach()` or you keep the whole graph, and `.remove()` or the hook fires on every subsequent forward pass and quietly overwrites your data.

8 Exercises

1. You tune the number of hidden units by trying eight values and keeping the one with the best test accuracy, then report that accuracy. What have you actually measured?

2. You run PCA on your whole dataset, then use `cross_val_score` on the components. Why are your scores too high?

3. Training accuracy is 0.61 and validation accuracy is 0.74. Name the most likely cause. (It is not that your model generalizes unusually well.)

4. Ten seeds give a mean accuracy of 0.82 with a standard deviation of 0.06. What is the standard error? How many seeds would you need to halve it?

5. `nn.Conv2d(3, 16, kernel_size=5, padding=2, stride=1)` applied to a $(8, 3, 64, 64)$ batch. Output shape? Number of parameters?

6. Your test-error-vs-width curve looks like meaningless noise with no U and no second descent. Give two things about how you plotted it that could cause that.

Answers

1. The best of eight noisy numbers, which is biased upward — you used the test set as a validation set. You have measured how well the best of eight models fits data you selected it on. The honest procedure: choose the width on a validation set, then evaluate that single model once on test.

2. PCA fitted on all the data has already seen every fold's held-out portion, so the "held-out" data influenced the features being scored. Fit the PCA inside each fold — `Pipeline` does this for you.

3. A bug. Most often the split is mismatched (validation is easier, or smaller and unstratified), or there is leakage, or dropout is on during training and off during evaluation and you are comparing the two regimes. Real generalization does not exceed training performance.

4. $0.06/\sqrt{10} \approx 0.019$. Halving it needs four times as many seeds — 40 — because the standard error falls as $1/\sqrt{n}$.

5. $H_{\text{out}} = (64 - 5 + 4)/1 + 1 = 64$, so $(8, 16, 64, 64)$. Parameters: $16 \cdot 3 \cdot 5 \cdot 5 + 16 = 1{,}216$.

6. (i) A linear x-axis, which compresses the whole first descent against the origin — use `xscale('log')`. (ii) Widths sampled too coarsely to bracket the interpolation threshold, so the peak falls between two of your points. Also acceptable: a single seed per width, so the curve is dominated by initialization noise.

Cheat sheet

You want	You write
a split that keeps class balance	`train_test_split(..., stratify=y)`
k-fold scores	`cross_val_score(clf, X, y, cv=5)`
no leakage in a fold	`Pipeline([('pca', PCA()), ('clf', ...)])`
the overfitting plot	`plt.plot(train); plt.plot(val)`
a double-descent curve	`plt.plot(widths, err); plt.xscale('log')`
standard error of a mean	`a.std(ddof=1) / np.sqrt(len(a))`
a conv layer	`nn.Conv2d(c_in, c_out, k, padding=p)`
its output shape	push `torch.zeros(1, c, h, w)` through it
one vision block	`Conv2d` → `ReLU` → `MaxPool2d(2)`
activations from a prefix	`net.features[:5](x)`
activations from anywhere	`register_forward_hook` (then `.remove()`)